



UNIVERSIDADE FEDERAL DE
SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
ALGORITMOS E ESTRUTURAS DE
DADOS I - 2026.1



MANUAL DE USO E DOCUMENTAÇÃO TÉCNICA TV TUBO SIMULATOR

1 Equipe

- **Arthur de Azevedo Grazzia** - (Slides: 15 a 17)
- **João Herman Souza de Araújo** - (Slides: 1 a 6)
- **João Vitor Vital Leão** - (Slides: 12 a 14)
- **Renato Veloso Pires Filho** - (Slides: 18 a 20)
- **Wilson Fernandes Carneiro Júnior** - (Slides: 7 a 11)

2 Objetivo

Simular o funcionamento de uma TV CRT usando Listas Simplesmente Encadeadas implementadas do zero. A grade de pixels é uma Multilista (array de listas encadeadas), onde cada lista representa uma scanline. O acesso sequencial obrigatório $O(N)$ da estrutura espelha o percurso físico do feixe de elétrons no tubo.

3 Stack

- **Backend:** Java 17, Javalin 6 (API REST), Maven.
- **Frontend:** React 18, Vite, Canvas API.
- **Infra:** Docker e Docker Compose.

4 Mapeamento Estrutura de Dados → Visual

Conceito AED	Implementação	Efeito Visual
Nó da lista	<code>Pixel.java</code>	Ponto luminoso na tela.
Lista encadeada	<code>Lista.java</code>	Scanline horizontal.
Multilista	<code>Multilista.java</code>	Grade completa: 40 linhas × 80 colunas (3200 nós).
Percurso $O(N)$	<code>for (current = current.next)</code>	Feixe avança pixel a pixel, sem saltos.
Decaimento	<code>luminance[i] *= 0.99</code> por frame	Fósforo apagando gradualmente a 60 fps.
Índice linear → 2D	$r = \lfloor i/80 \rfloor, c = i \bmod 80$	Posição do feixe mapeada para (lista, nó).

Tabela 1: Relação entre estrutura de dados e efeito visual.

5 Implementação — Backend

5.1 `Pixel.java` — Nó da lista

```
1 public class Pixel {
2     int value; // brilho do fosforo (0-255)
3     Pixel next; // proximo pixel da scanline
4
5     Pixel(int value) { this.value = value; }
6 }
```

5.2 `Lista.java` — Travessia $O(N)$

Sem acesso direto por índice: o algoritmo percorre todos os nós anteriores ao alvo.

```
1 public void setPixel(int index, int value) {
2     Pixel current = head;
3     for (int i = 0; i < index; i++)
4         current = current.next;
5     current.value = value;
6 }
```

5.3 `Multilista.java` — Grade bidimensional

Array de Listas: acesso à linha em $O(1)$, acesso ao pixel em $O(N)$.

```
1 public Multilista(int rows, int cols) {
2     this.rows = new Lista[rows];
3     for (int i = 0; i < rows; i++) {
4         this.rows[i] = new Lista();
5         for (int j = 0; j < cols; j++)
6             this.rows[i].addPixel(0);
7     }
8 }
```

5.4 Main.java — API REST

O endpoint `/api/state` serializa a Multilista para JSON a cada requisição.

```
1 app.get("/api/state", ctx -> {
2   int[][] grid = new int[ROWS][COLS]; // 40 x 80
3   for (int r = 0; r < ROWS; r++)
4     for (int c = 0; c < COLS; c++)
5       grid[r][c] = tela.getPixel(r, c);
6   ctx.json(grid);
7 });
```

6 Implementação — Frontend

6.1 App.jsx — Sincronização com o backend

```
1 const [grid, setGrid] = useState([]);
2
3 const fetchGrid = () => {
4   fetch('/api/state')
5     .then(r => r.json())
6     .then(setGrid); // atualiza o canvas automaticamente
7 }
```

6.2 PixelGrid.jsx — Motor gráfico (Canvas API)

Renderização via Canvas evita manipulação do DOM para 3200 pixels. O decaimento do fósforo e o mapeamento do índice linear para a grade da Multilista são feitos a cada frame.

```
1 const DECAY = 0.99;
2
3 // Decai o fosforo de todos os pixels
4 for (let i = 0; i < total; i++) {
5   s.luminance[i] *= DECAY;
6   if (s.luminance[i] < 1) s.luminance[i] = 0;
7 }
8
9 // Mapeia indice linear para (lista, no) da Multilista
10 const r = Math.floor(i / cols);
11 const c = i % cols;
12 s.luminance[i] = s.grid[r][c];
13
14 rafId = requestAnimationFrame(frame); // 60 fps
```

6.3 DataPanel.jsx — Visualização da Multilista

Exibe em tempo real uma barra por lista (L00–L39) proporcional ao número de nós ativos, e o contador global de nós ativados sobre 3200.

7 Manual de Operação

7.1 Pré-requisito

Instalar o **Docker Desktop** (docker.com/get-started). Não é necessário instalar Java, Node.js ou qualquer outra dependência — tudo é compilado e executado dentro dos contêineres.

7.2 Executar a aplicação

No terminal, dentro da pasta do projeto:

```
1 docker compose up --build -d
2 # Acesse: http://localhost:8080
```

Para encerrar:

```
1 docker compose down
```

7.3 Recursos da Interface

7.3.1 Desenho livre

Clique e arraste na tela para desenhar. O modo (acender/apagar) é definido pelo estado do primeiro pixel tocado.

7.3.2 Padrões disponíveis

- **CORAÇÃO** — equação implícita $(x^2 + y^2 - 1)^3 - x^2y^3 \leq 0$.
- **ÁRVORE** — três triângulos sobrepostos com tronco.
- **ESTRELA** — polígono de 5 pontas via ray-casting.
- **SÓLIDO** — todos os 3200 nós ativos.
- **CÍRCULO** — disco central por distância euclidiana.
- **GRADIENTE** — intensidade 0–255 da esquerda para a direita.
- **VERTICAL** — colunas pares ativas.
- **HORIZONTAL** — linhas pares ativas (scanlines visíveis).
- **GRADE** — malha a cada 4 posições.
- **DIAGONAL** — linhas diagonais a cada 8 posições.
- **XADREZ** — pixels alternados.
- **BORDA** — apenas o perímetro da grade.
- **ONDA** — curva senoidal coluna a coluna via $\sin$.

7.3.3 VER ESTRUTURA

Abre o `DataPanel`: barras por lista mostrando nós ativos e contador global.

7.3.4 LIMPAR

Envia `POST /api/clear`: zera todos os 3200 nós da Multilista.

8 Vídeo de Apresentação

O vídeo de apresentação do trabalho, bem como os demais arquivos do projeto, estão disponíveis na pasta compartilhada da equipe:

https://drive.google.com/drive/folders/1U87qw185C8NKVX_4gURCESAmhsiYlvxn